

Módulo 4 – Capítulo 02 – Sección 01

Patrones de Arquitectura para IA

Antes de construir cualquier componente de un sistema de IA, el arquitecto debe responder una pregunta que determinará la forma global de todo lo que venga después: ¿cómo se organiza el sistema? Esta pregunta no tiene respuesta tecnológica única. Tiene respuestas arquitectónicas, y esas respuestas han sido destiladas durante décadas de práctica en ingeniería de software en un conjunto de patrones reconocibles: estructuras de organización que resuelven problemas recurrentes de una forma que ha sido probada en producción.

Los patrones de arquitectura no son fórmulas. Son vocabulario compartido que permite a los equipos comunicarse con precisión sobre cómo está organizado un sistema, y herramientas conceptuales que permiten al arquitecto razonar sobre las consecuencias de una decisión de organización antes de implementarla. Cuando un equipo dice "vamos a construir esto como un monolito" o "esto requiere una arquitectura orientada a eventos", está comunicando una decisión de diseño que tiene implicaciones conocidas sobre escalabilidad, complejidad operativa, facilidad de despliegue y velocidad de desarrollo.

En el contexto de sistemas de IA, la elección del patrón arquitectónico tiene consecuencias que van más allá de las que tendría en sistemas tradicionales. Los sistemas de IA tienen características peculiares que los patrones deben acomodar: las llamadas a modelos de lenguaje son costosas y lentas comparadas con las operaciones de negocio típicas; la calidad de las respuestas depende de la calidad de los datos de contexto; los pipelines de inferencia tienen requisitos de latencia distintos a los pipelines de procesamiento de datos; y los costos de operación escalan de manera no lineal con el uso. Un patrón que funciona perfectamente para una API de e-commerce puede ser completamente inadecuado para un sistema RAG con cientos de documentos actualizándose diariamente.

Este capítulo examina los tres patrones principales que los arquitectos de IA encuentran con mayor frecuencia — monolito, microservicios y arquitecturas basadas en eventos — y luego aborda la decisión más fundamental de todo el módulo: cómo elegir entre enriquecer el

modelo (fine-tuning) y enriquecer el contexto (RAG), dos estrategias con implicaciones arquitectónicas radicalmente distintas.

El principio que guía toda la selección de patrones es uno que el Capítulo 01 ya estableció: la decisión arquitectónica debe estar justificada por las necesidades del negocio, el volumen de uso, el presupuesto, el tamaño del equipo y los requisitos no funcionales del sistema. No por la popularidad de una tecnología en el ecosistema, ni por la preferencia personal del equipo de desarrollo. Un arquitecto que no puede articular por qué eligió un patrón en términos de necesidades del negocio probablemente no eligió el patrón correcto.

Las secciones que siguen examinan cada patrón con sus beneficios específicos, sus limitaciones reales y los casos de uso en los que cada uno es la elección más razonable para un sistema de IA en producción.

Módulo 4 – Capítulo 02 – Sección 02

Arquitectura Monolítica

La arquitectura monolítica tiene mala reputación en el ecosistema tecnológico contemporáneo, asociada en el imaginario colectivo con sistemas heredados, lentos y difíciles de mantener. Esa reputación es injusta y, en muchos contextos de sistemas de IA, directamente contraproducente. Entender cuándo un monolito es la decisión correcta — y cuándo deja de serlo — es una de las competencias más prácticas de un arquitecto.

Un monolito es una aplicación en la que todos los componentes del sistema se despliegan como una unidad única. El pipeline de ingesta de documentos, la lógica de recuperación, el servicio de inferencia, la autenticación y la interfaz de usuario viven en el mismo proceso, en el mismo servidor y se despliegan juntos. Esa unidad produce una lista de beneficios concretos que los sistemas más complejos no tienen.

Beneficios del monolito para sistemas de IA:

- **Simplicidad operativa:** un único proceso para monitorear, desplegar y escalar. No hay comunicación de red entre servicios, no hay problemas de consistencia entre versiones de distintos componentes, no hay latencia adicional por llamadas entre servicios.
- **Velocidad de iteración:** los cambios en cualquier parte del sistema se despliegan en una única operación. En un producto de IA en fase de validación de mercado, donde el equipo modifica la estrategia de prompting, los criterios de retrieval y la lógica de negocio varias veces por semana, la simplicidad del ciclo de despliegue tiene un valor real.
- **Coherencia de datos:** todos los componentes acceden a las mismas bases de datos, con las mismas credenciales y las mismas reglas de consistencia. No hay sincronización entre bases de datos de servicios distintos.
- **Observabilidad unificada:** todos los logs, trazas y métricas comparten el mismo contexto de ejecución. El debugging de un flujo completo — desde la solicitud del usuario hasta la respuesta del modelo — no requiere correlacionar trazas entre múltiples servicios.

Limitaciones del monolito:

- **Escalabilidad no granular:** si el servicio de inferencia requiere GPU y el servicio de autenticación no, ambos deben desplegarse en el mismo servidor, lo que implica pagar por GPU también para los componentes que no la necesitan.
- **Acoplamiento involuntario:** sin fronteras de servicio explícitas, los equipos tienden a crear dependencias no intencionales entre módulos, haciendo que el sistema sea progresivamente más difícil de modificar.
- **Despliegues totales:** cualquier cambio, por pequeño que sea, requiere redesplegar el sistema completo, lo que incrementa el riesgo de cada despliegue.
- **Punto único de fallo:** un error en cualquier componente puede afectar la disponibilidad de todo el sistema.

Casos de uso donde el monolito es la elección correcta:

- MVP o producto en validación de mercado con menos de 50 usuarios activos.
- Equipo de desarrollo pequeño (1-5 personas) sin experiencia operativa en sistemas distribuidos.
- Presupuesto limitado que no justifica la complejidad operativa de múltiples servicios.
- Sistema de IA interno con carga predecible y baja, como un asistente de documentación para uso del equipo de ingeniería.

El patrón que sugiere comenzar con un monolito y extraer servicios cuando sea necesario ("Modular Monolith") es una estrategia arquitectónica reconocida y respetada. Sam Newman, autor de *Building Microservices*, recomienda explícitamente comenzar con un monolito para sistemas nuevos, porque los límites del servicio se descubren operando el sistema, no en una reunión de diseño inicial.

Nota del Arquitecto: El error más común que cometen los equipos con experiencia en microservicios es aplicar ese patrón a un sistema nuevo desde el primer día, convencidos de que es la "mejor práctica". El resultado habitual es un sistema distribuido en el que la complejidad operativa supera la capacidad del equipo, los servicios están mal divididos porque los límites del dominio no estaban claros, y la velocidad de desarrollo es una fracción de lo que sería con un monolito bien estructurado.

La transición del monolito al sistema distribuido debe estar motivada por necesidades reales: escalado diferencial de componentes, equipos independientes que necesitan ciclos de despliegue autónomos, o requisitos de resiliencia que el monolito no puede satisfacer. La

siguiente sección examina el patrón al que ese monolito eventualmente puede evolucionar: microservicios e IA.

Módulo 4 – Capítulo 02 – Sección 03

Microservicios e IA

Cuando un sistema de IA crece en complejidad, en volumen de uso y en tamaño del equipo de desarrollo, las limitaciones del monolito se vuelven obstáculos concretos. El servicio de inferencia necesita escalar de forma independiente del servicio de negocio. El equipo de ML necesita desplegar cambios en el pipeline de RAG sin esperar al ciclo de despliegue del equipo de frontend. El servicio de embeddings necesita ejecutarse en hardware con GPU mientras el servicio de autenticación no. Es en ese momento cuando la arquitectura de microservicios comienza a producir valor real, no antes.

En una arquitectura de microservicios aplicada a sistemas de IA, los componentes del sistema se organizan en servicios independientes con fronteras bien definidas, contratos de API estables y ciclos de despliegue autónomos. Una configuración típica para un sistema RAG productivo puede incluir un servicio de ingesta responsable de procesar documentos nuevos o actualizados, un servicio de retrieval que opera sobre la base vectorial, un servicio de inferencia que gestiona las llamadas al modelo de lenguaje, un servicio de composición de contexto que combina el resultado del retrieval con la solicitud del usuario, y un API gateway que autentica las solicitudes y las enruta hacia el servicio correspondiente.

Beneficios de los microservicios en sistemas de IA:

- **Escalado granular:** el servicio de inferencia puede escalarse horizontalmente durante los picos de demanda sin incrementar los recursos del servicio de ingesta, que opera de manera asíncrona y con carga predecible.
- **Despliegues independientes:** el equipo de ML puede actualizar la estrategia de retrieval y el modelo de embeddings sin afectar al equipo de backend que mantiene la lógica de negocio.
- **Isolation de fallos:** si el servicio de inferencia del modelo comercial experimenta una degradación de la API externa, el resto del sistema puede continuar operando. Un circuit breaker puede activar un fallback hacia un modelo local o una respuesta cacheada.
- **Heterogeneidad tecnológica:** el servicio de inferencia puede estar implementado en Python con acceso a GPU, mientras el servicio de autenticación opera en Go con requisitos

de latencia sub-milisegundo.

Limitaciones específicas para sistemas de IA:

- **Latencia de red acumulativa:** si una solicitud de usuario pasa por el API gateway, el servicio de composición de contexto, el servicio de retrieval y el servicio de inferencia de forma secuencial, la latencia de red de cada salto se acumula. En sistemas con requisitos de respuesta inferior a dos segundos, este presupuesto de latencia puede resultar ajustado.
- **Complejidad operativa:** múltiples servicios implican múltiples pipelines de CI/CD, múltiples configuraciones de logging y múltiples puntos de observabilidad que deben ser correlacionados mediante trazas distribuidas.
- **Service mesh y coordinación:** en sistemas con muchos servicios, la coordinación de la comunicación entre ellos puede requerir un service mesh (como Istio o Linkerd), que agrega una capa adicional de infraestructura.

Patrones específicos de microservicios para sistemas de IA:

- **Sidecar de observabilidad:** un contenedor auxiliar que acompaña al servicio de inferencia y captura todas las llamadas al modelo, los tokens consumidos y los tiempos de respuesta sin modificar el código del servicio principal. Este patrón permite instrumentar sin intrusión.
- **API contract entre servicio de negocio y servicio de RAG:** definir un esquema estricto para la solicitud de contexto (qué filtros se aplican, qué campos se incluyen en los metadatos de los chunks, qué nivel de confianza mínimo se acepta) desacopla al equipo de negocio del equipo de ML de forma que cada uno puede evolucionar su lado del contrato independientemente.
- **Circuit breaker para llamadas a LLM externos:** cuando la API de un proveedor externo experimenta alta latencia o errores, el circuit breaker abre el circuito y activa una respuesta alternativa — un mensaje de degradación elegante o una llamada a un modelo de respaldo — en lugar de propagar los errores a todos los usuarios.
- **Dedicated pod de inferencia en Kubernetes:** el servicio de inferencia se despliega en nodos con GPU dedicados, aislados de los nodos de CPU del resto de los servicios, permitiendo una política de escalado y un presupuesto de costos independientes.

Nota del Arquitecto: El error de diseño más frecuente al adoptar microservicios en sistemas de IA es dividir los servicios por tecnología en lugar de por dominio funcional. Crear un "servicio de Python" y un "servicio de Node.js" no produce los

beneficios de los microservicios. Crear un "servicio de ingesta" y un "servicio de recuperación" con responsabilidades de dominio claras sí los produce.

Cuando la carga de trabajo tiene una naturaleza asíncrona — ingesta de documentos, procesamiento de lotes, evaluación periódica de calidad — los microservicios pueden combinarse con el patrón de arquitectura basada en eventos que examina la sección siguiente.

Módulo 4 – Capítulo 02 – Sección 04

Arquitecturas Basadas en Eventos

No todas las operaciones de un sistema de IA tienen el mismo perfil de temporalidad. Cuando un usuario hace una pregunta, espera una respuesta en segundos: esa interacción es síncrona por naturaleza. Pero cuando un nuevo documento se incorpora al knowledge base de un sistema RAG, el proceso de extracción, limpieza, chunking, generación de embeddings y almacenamiento puede tomar minutos y no necesita completarse antes de que el usuario reciba ninguna confirmación inmediata. Esta diferencia de temporalidad es exactamente el problema que resuelven las arquitecturas basadas en eventos.

En una arquitectura basada en eventos, los componentes del sistema no se llaman directamente entre sí. En cambio, publican eventos en una cola o bus de mensajes, y otros componentes se suscriben a esos eventos para procesarlos de forma asíncrona. Cuando un editor de contenidos sube un nuevo documento al repositorio corporativo, ese acto publica un evento `documento.creado` en una cola (por ejemplo, AWS SQS, Azure Service Bus, o Apache Kafka). El servicio de ingesta de RAG está suscrito a ese evento y procesa el documento en segundo plano, sin bloquear ninguna operación del usuario. Cuando el documento ha sido procesado y sus embeddings están disponibles en la base vectorial, el servicio publica un evento `documento.indexado`, que puede disparar una notificación al editor o un proceso de validación de calidad.

Beneficios específicos para sistemas de IA:

- **Desacoplamiento temporal:** el productor del evento no necesita esperar al consumidor. El editor que carga un documento no espera a que el pipeline de ingesta termine. El servicio de negocio que actualiza la información de un producto no espera a que el vector store se actualice.
- **Resiliencia por reintentos:** si el servicio de embeddings está temporalmente indisponible, el mensaje permanece en la cola y el servicio lo procesará cuando recupere disponibilidad. No se pierden documentos por fallos transitorios.
- **Escalado elástico del pipeline de ingesta:** cuando se cargan cien documentos simultáneamente (por ejemplo, durante una migración de contenidos), el pipeline de

ingesta puede escalar horizontalmente leyendo múltiples mensajes de la cola en paralelo, sin afectar la latencia de las consultas de usuarios.

- **Procesamiento por lotes:** los eventos pueden acumularse y procesarse en lotes durante períodos de baja demanda, reduciendo el costo de las llamadas a APIs de embeddings que cobran por volumen.

Casos de uso en sistemas de IA:

- Actualización del knowledge base de un sistema RAG cuando documentos fuente cambian (repositorio de documentos → cola → pipeline de ingesta → base vectorial).
- Evaluación periódica de calidad de respuestas: un proceso nocturno extrae conversaciones del día, las procesa con métricas RAGAS y publica resultados en un dashboard de calidad.
- Pipeline de fine-tuning disparado por eventos: cuando el volumen de conversaciones humanas anotadas supera un umbral, se dispara automáticamente un job de fine-tuning.
- Notificaciones asíncronas en sistemas de agentes: cuando un agente completa una tarea de larga duración (análisis de un informe de 200 páginas), el resultado se publica como evento en lugar de mantener una conexión HTTP abierta.

Limitaciones que el arquitecto debe gestionar:

- **Consistencia eventual:** los componentes que leen datos procesados asincrónicamente pueden leer datos desactualizados durante el período entre el evento y su procesamiento. Para sistemas RAG, esto significa que puede haber una ventana de tiempo en la que un documento actualizado aún no está disponible en la base vectorial.
- **Dificultad de debugging:** rastrear un flujo de datos a través de múltiples consumidores de eventos requiere correlación de trazas distribuidas, lo cual es más complejo que rastrear una llamada síncrona.
- **Complejidad operativa:** la cola de mensajes es una infraestructura adicional que debe ser monitoreada, con métricas propias como profundidad de cola, tasa de mensajes sin procesar y latencia de procesamiento.

Nota del Arquitecto: Las arquitecturas basadas en eventos son el mecanismo natural para los pipelines de ingesta de sistemas RAG en producción. Pero el error que cometen muchos equipos es usarlas también para la ruta crítica de consultas del usuario, donde la sincronidad es un requisito del producto. Un sistema RAG bien diseñado usa eventos para la ingesta (asíncrona, tolerante a latencia) y llamadas síncronas para las consultas de usuario (donde la latencia es un atributo de calidad percibida).

Las tres secciones anteriores han presentado los patrones de arquitectura más frecuentes en sistemas de IA. La sección siguiente aborda la pregunta más fundamental de todo el capítulo: cómo elegir entre ellos, y en particular, cómo tomar la decisión que precede a cualquier elección de patrón de infraestructura — si enriquecer el modelo o enriquecer el contexto.

Módulo 4 – Capítulo 02 – Sección 05

Fine-tuning vs RAG vs Prompt Engineering: La Decisión Arquitectónica Fundamental

Antes de seleccionar entre monolito, microservicios o eventos, el arquitecto de IA debe responder una pregunta de mayor jerarquía: ¿dónde vive el conocimiento que el sistema necesita para responder correctamente? Esta decisión — enriquecer el modelo, enriquecer el contexto o enriquecer las instrucciones — tiene implicaciones arquitectónicas que afectan el costo, la complejidad operativa, la frecuencia de actualización del conocimiento y la trazabilidad de las respuestas. Es posiblemente la decisión de mayor impacto en todo el diseño de un sistema de IA.

Las tres estrategias principales son prompt engineering, RAG (Retrieval-Augmented Generation) y fine-tuning. No son mutuamente excluyentes, pero cada una resuelve un tipo distinto de problema y viene con un perfil de costo y complejidad muy diferente.

Prompt Engineering — el conocimiento está en las instrucciones

El prompt engineering consiste en dotar al modelo de conocimiento a través de instrucciones, ejemplos y contexto incluidos directamente en el prompt del sistema. Es la estrategia más simple arquitectónicamente: no requiere infraestructura adicional, no tiene costo de entrenamiento y produce resultados inmediatos. Es la elección correcta cuando el dominio de conocimiento es estable, bien delimitado y puede describirse en unas pocas páginas. Un asistente de clasificación de tickets de soporte que debe seguir una taxonomía de veinte categorías fijas puede implementarse exclusivamente con prompt engineering. Un asistente que debe responder preguntas sobre un catálogo de diez mil productos que cambia diariamente no puede.

RAG — el conocimiento está en el sistema

RAG enriquece el contexto del modelo en tiempo de consulta: recupera fragmentos relevantes del knowledge base y los incluye en el prompt antes de enviar la solicitud al modelo. El modelo no necesita "saber" nada: lo busca en el momento en que lo necesita. Esta estrategia

es correcta cuando el conocimiento es voluminoso (no cabe en el prompt del sistema), cambia con frecuencia (se actualiza semanalmente o diariamente), o tiene requisitos de trazabilidad (el arquitecto necesita saber exactamente qué fuente informó cada respuesta).

Los costos de RAG son principalmente operativos: infraestructura de la base vectorial, pipeline de ingesta, costo de generación de embeddings para cada documento nuevo. El costo de entrenamiento es cero, pero el costo de operación es continuo y escala con el volumen de documentos y el número de consultas.

Fine-tuning — el conocimiento está en el modelo

El fine-tuning adapta los pesos del modelo mediante entrenamiento adicional sobre un conjunto de datos específico del dominio. El modelo "aprende" el conocimiento y puede responder sin necesidad de recuperar contexto externo. Es la estrategia correcta cuando el sistema necesita que el modelo adopte un estilo de respuesta muy específico que el prompting no logra (por ejemplo, seguir un formato médico estricto), cuando la latencia de la recuperación de contexto es inaceptable, o cuando el conocimiento es estático y altamente especializado.

El fine-tuning tiene costos de entrenamiento significativos (compute, tiempo de ingeniería para preparar el dataset de entrenamiento), costos de evaluación (se necesita un proceso riguroso de evaluación antes de desplegar el modelo fino), y un costo de actualización alto: cada vez que el conocimiento cambia, el proceso de fine-tuning debe repetirse.

Dimensión	Prompt Engineering	RAG	Fine-tuning
Costo de setup	Muy bajo	Medio-alto	Alto
Costo de actualización	Muy bajo	Bajo	Alto
Capacidad de conocimiento	Limitada (ventana de contexto)	Alta (base vectorial ilimitada)	Alta (pesos del modelo)
Trazabilidad	Limitada	Alta (chunks recuperados visibles)	Baja (conocimiento opaco)
Latencia de respuesta	Baja	Media (retrieval + inferencia)	Baja
Frescura del conocimiento	Inmediata (cambio de prompt)	Alta (ingesta continua)	Baja (requiere reentrenamiento)

Cuándo ninguna estrategia es suficiente por sí sola

Existen casos en los que ninguna de las tres estrategias individualmente resuelve el problema: el sistema necesita razonar sobre conocimiento dinámico (RAG), adaptarse a un estilo de respuesta específico (fine-tuning), y ejecutar acciones en sistemas externos (agentes). En esos casos, la arquitectura correcta combina las tres estrategias en capas: el modelo fino provee la base de comportamiento, el RAG provee el conocimiento actualizado, y las herramientas del agente proveen las capacidades de acción. El Capítulo 04 examina esa arquitectura en detalle.

Nota del Arquitecto: La pregunta que el arquitecto debe hacer antes de decidir entre RAG y fine-tuning no es "¿cuál produce mejores respuestas?" sino "¿con qué frecuencia cambia el conocimiento y quién es responsable de mantenerlo actualizado?" Si el conocimiento cambia mensualmente y hay un equipo dedicado a curar el dataset de entrenamiento, el fine-tuning puede ser viable. Si el conocimiento cambia diariamente y los responsables de contenido no tienen experiencia técnica, el RAG con un pipeline de ingesta automatizado es la única opción operacionalmente sostenible.

La decisión entre estas estrategias debe tomarse antes de cualquier otra decisión arquitectónica, porque determina qué tipo de infraestructura necesita el sistema, qué equipos deben estar involucrados en el mantenimiento y cuál será el presupuesto operativo a largo plazo. La sección siguiente cierra el capítulo con un marco de criterios para seleccionar entre los patrones de infraestructura estudiados.

Módulo 4 – Capítulo 02 – Sección 06

Resumen

Este capítulo desarrolló el vocabulario arquitectónico fundamental que el arquitecto de IA necesita para tomar las decisiones de más alto nivel de un sistema: cómo está organizado y dónde vive el conocimiento que el sistema utiliza para responder.

El patrón monolítico, frecuentemente subestimado, es la elección correcta para sistemas en etapa de validación, equipos pequeños y presupuestos limitados. Su ventaja — simplicidad operativa — se convierte en limitación cuando el sistema escala o cuando diferentes componentes requieren ciclos de despliegue y recursos de hardware distintos. El monolito no es el enemigo de los sistemas de IA; es el punto de partida adecuado para muchos de ellos.

Los microservicios resuelven las limitaciones del monolito a costa de una complejidad operativa significativa. Los beneficios — escalado granular, despliegues independientes, isolation de fallos — justifican esa complejidad cuando el sistema ha demostrado su valor en producción y los límites del dominio están suficientemente claros. En sistemas de IA, los patrones específicos de microservicios como el sidecar de observabilidad y el circuit breaker para LLMs externos son mecanismos que protegen al sistema de los comportamientos particulares de las APIs de modelos de lenguaje.

Las arquitecturas basadas en eventos son el mecanismo natural para los pipelines asíncronos de sistemas de IA: ingesta de documentos, evaluación periódica de calidad, procesamiento de lotes y pipelines de fine-tuning disparados por umbrales de datos. Su valor está en el desacoplamiento temporal: el productor no espera al consumidor, lo que incrementa la resiliencia y la capacidad de escalar cada etapa independientemente.

La decisión de mayor jerarquía — prompt engineering, RAG o fine-tuning — debe tomarse antes de cualquier decisión de infraestructura, porque determina el tipo de sistema que se construirá. El prompt engineering es simple pero limitado en volumen y frecuencia de actualización. El RAG es el patrón correcto para conocimiento voluminoso y dinámico, con alta trazabilidad. El fine-tuning es adecuado para adaptar el comportamiento del modelo

cuando el conocimiento es estático y especializado, pero su costo de actualización lo hace inadecuado para dominios que cambian frecuentemente.

El arquitecto correcto no es el que conoce todos los patrones, sino el que puede articular con precisión por qué elige uno sobre otro en un contexto dado. Esa articulación — las necesidades del negocio, el presupuesto, el equipo, el volumen de uso, la frecuencia de cambio del conocimiento — es exactamente lo que diferencia una decisión arquitectónica de una preferencia tecnológica.

El Capítulo 03 aplica estos principios al patrón de mayor adopción en sistemas de IA empresariales actuales: la arquitectura RAG en producción, con toda su infraestructura de ingesta, recuperación, generación y operación continua.

"La arquitectura de software no trata de las herramientas que usas. Trata de las decisiones que tomas antes de usar cualquier herramienta."

— Martin Fowler, autor de *Patterns of Enterprise Application Architecture*